

Klassen mit veränderlichen Eigenschaften

1 Motivation

Mit records lassen sich Klassen sehr einfach definieren.

Eine starke Einschränkung von records ist aber, dass deren Eigenschaften nach dem Erzeugen nicht mehr geändert werden können.

```
1 record Student(String name, int age){}
```

```
1 var pana = new Student("Pana" , 17);
```

Der Student pana hat das Alter 17. Dies kann nicht geändert werden. Man kann lediglich einen neuen Student mit einem anderen Alter erzeugen und den alten Student überschreiben.

```
1 var pana = new Student("Pana" , 18);
```

Bei größeren Objekten mit vielen Eigenschaften ist das jedoch sehr ineffizient.

2 Klassen mit class definieren

Klassen mit veränderlichen Eigenschaften können in Java mit dem Schlüsselwort `class` definiert werden. Alle Eigenschaften werden im Klassenkörper definiert.

```
class MutableStudent{  
    String name;  
    int age;  
}
```

3 Konstruktoren definieren

Alle Konstruktoren müssen selbst geschrieben werden. Konstruktoren werden ähnlich wie Methoden definiert. Der Name des Konstruktors stimmt immer mit dem Namen der Klasse überein. Ein Rückgabotyp wird nicht angegeben.

```
class MutableStudent{
    String name;
    int age;

    public MutableStudent(String pName, int pAge){
        name = pName;
        age = pAge;
    }
}
```

Zwischen den geschweiften Klammern kann jeder beliebige Java-Code stehen. In der Regel werden aber nur Werte für die Eigenschaften der Klasse gesetzt.

Hier wird

- die Eigenschaft name auf den übergebenen Wert pName gesetzt
- die Eigenschaft age auf den übergebenen Wert pAge gesetzt

Wenn die Argumente des Konstruktors dieselben Namen wie die Eigenschaften der Klasse haben, muss mit `this.eigenschaft` auf eine Eigenschaft zugegriffen werden.

```
...
public MutableStudent(String name, int age){
    this.name = name;
    this.age = age;
}
```

```
1 var pana = new MutableStudent("Pana", 17);
```

4 Methoden

Methoden können wie bei Records definiert werden. Es ist jetzt aber möglich, die Eigenschaften der Klasse in Methoden zu ändern.

```
class MutableStudent{
    String name;
    int age;
    ...
    void getOlder(){
        age = age + 1;
    }
}
```

Die Methode `getOlder` erhöht das Alter eines `MutableStudent`. Dabei wird **kein** neues Objekt erzeugt.

```

1 var luca = new MutableStudent("Luca", 18);
2 luca.getOlder();
3 luca.age;

```

19

In der Regel werden Eigenschaften als `private` und Konstruktoren sowie Methoden als `public` deklariert.

```

class MutableStudent{
    private String name;
    private int age;

    public MutableStudent(String name, int age){
        this.name = name;
        this.age = age;
    }

    public void getOlder(){
        age = age + 1;
    }
}

```

5 Getter und Setter

Um trotzdem auf `private` Eigenschaften zuzugreifen, werden Methoden definiert, die den Wert einer Eigenschaft zurückgeben.

```

...
public int getAge(){
    return age;
}
...

```

```

1 var luca = new MutableStudent("Luca", 18);
2 luca.getAge();

```

18

oder ändern.

```

...
    public void setAge(int age){
        this.age = age;
    }
    ...
}

```

```

1 luca.setAge(19);
2 luca.getAge();

```

19

6 Invalide Werte durch Fehlermeldungen im Setter und Konstruktor verhindern

Wenn ein Setter definiert ist, kann es trotzdem sein, dass ein privates Attribut auf einen ungültigen Wert gesetzt wird.

```

1 var matti = new MutableStudent("Matti", 17);
2 matti.setAge(-3);
3 matti.getAge();

```

-3

Wir können das verhindern, wenn wir eine Fehlermeldung werfen, wenn dem Setter ein ungültiger Wert übergeben wird.

```

...
    public void setAge(int age){
        if (age < 0){
            throw new IllegalArgumentException("age must be positive");
        }
        this.age = age;
    }
}

```

Leider ist es immer noch möglich, mit dem Konstruktor Objekte mit ungültigen Eigenschaften zu erzeugen.

```

1 var matti = new MutableStudent("Matti", -17);
2 matti.getAge();

```

-17

Deshalb sollte die Überprüfung auch im Konstruktor genutzt werden.

```

public MutableStudent(String name, int age){
    if (age < 0){
        throw new IllegalArgumentException("age must be positive");
    }
    this.name = name;
    this.age = age;
}

```

7 Eigenschaften im Klassenkörper initialisieren

Wir können Eigenschaften auch direkt im Klassenkörper, statt im Konstruktor initialisieren.

```

1 class Cat{
2     private String name;
3     public int lives = 7;
4
5     Cat(String name){
6         this.name = name;
7     }
8 }

```

```

1 var garfield = new Cat("Garfield");
2 garfield.lives;

```

7

Bei jedem Objekt der Klasse Cat hat die Eigenschaft lives nach dem Erzeugen den Wert 7.

```

1 var catmando = new Cat("Catmando");
2 catmando.lives;

```

7